LECTURE SERIES · CSE 2206 DATABASE SYSTEMS · WEEK 7

Transactions, Rollback & Savepoints

From Autocommit Accidents to All-or-Nothing Changes in PostgreSQL



Md. Rakib Trofder & Saad Ahmed

Lecturers, Department of CSE, Bangladesh Army University of Science & Technology

BEGIN · COMMIT · ROLLBACK · SAVEPOINT · RELEASE

TODAY'S ROADMAP

Week 7 Agenda

Until now every statement was saved the instant it ran. This week we group statements into transactions — so we can check our work, undo mistakes, and never leave a job half-finished.

01

The Autocommit Problem

Why instant saves are dangerous

02

COMMIT & ROLLBACK

Keep or throw away a set of changes

03

Atomicity

All-or-nothing, even when errors occur

04

Isolation

Uncommitted work stays private

05

Savepoints

Partial undo inside a transaction

06

Lab Task

Hands-on inventory restocking exercise

WHY WE NEED TRANSACTIONS

The Problem: Autocommit

Without BEGIN, PostgreSQL runs in autocommit mode — every statement is saved permanently the moment it finishes. There is no undo button, and a job that stops halfway stays half-done.

Scene 1 — A mistake you cannot undo

A 500 Tk bonus meant only for Jamal (C-303) — but no WHERE.

```
-- Meant: ... WHERE acc_no = 'C-303';
UPDATE account
   SET balance = balance + 500;
-- Updated Rows: 5   (already saved!)

SELECT SUM(balance) FROM account;
-- 8000.00 -> 10500.00

-- The only fix: repair it by hand
UPDATE account
   SET balance = balance - 500;
-- What if it had been SET balance = 0 ?
```

Scene 2 — Half-finished work

Transfer 500 Tk A-101 → B-202, with a typo (letter O).

```
-- Step 1: take 500 from Rahim
UPDATE account SET balance = balance - 500
 WHERE acc_no = 'A-101';    -- saved!

-- Step 2: typo 'B-2O2' (letter O, not zero)
UPDATE account SET balance = balance + 500
 WHERE acc_no = 'B-2O2';
-- Updated Rows: 0, and NO error

SELECT SUM(balance) FROM account;
-- 7500.00 -> 500 Tk vanished!
```

THE SOLUTION

Transaction Control Commands

A transaction groups statements so they succeed together or fail together. Nothing is permanent until you say so.

Command	What it does	Why it helps
BEGIN	Starts a transaction — from now on nothing is permanent	Try a change before keeping it
COMMIT	Makes every change since BEGIN permanent	Saves all steps together (Scene 2)
ROLLBACK	Throws away every change since BEGIN	Undoes a mistake (Scene 1)
SAVEPOINT name	Places a bookmark inside the open transaction	Marks a safe point to return to
ROLLBACK TO SAVEPOINT	Undoes only the work after the bookmark	Partial undo — no retyping

KEEP IT OR UNDO IT

COMMIT & ROLLBACK

After BEGIN every change is tentative. Check the result with SELECT, then decide: COMMIT keeps everything, ROLLBACK throws everything away.

Example 1 — Undo a disaster with ROLLBACK

Every account accidentally gets 5,00,000 Tk.

```
BEGIN;
UPDATE account
   SET balance = balance + 500000;

SELECT * FROM account
 ORDER BY acc_no;   -- every balance is huge!

ROLLBACK;

SELECT * FROM account
 ORDER BY acc_no;
-- restored: A-101 1000 ... E-505 3000
```

Example 2 — A correct transfer with COMMIT

Move 500 Tk from Rahim (A-101) to Karim (B-202).

```
BEGIN;
UPDATE account SET balance = balance - 500
 WHERE acc_no = 'A-101';
UPDATE account SET balance = balance + 500
 WHERE acc_no = 'B-202';

-- Check before deciding
SELECT SUM(balance) FROM account;
-- 8000.00 -> correct

COMMIT;   -- now it is permanent
```

ALL OR NOTHING

Atomicity

Either every step is saved or none is. A real program writes both paths — COMMIT if all steps worked, ROLLBACK if anything failed — and if the server crashes before COMMIT, PostgreSQL undoes the unfinished work itself.

Example 1 — Catch the typo before it is saved

The same 'B-2O2' typo — this time inside a transaction.

```
BEGIN;
UPDATE account SET balance = balance - 500
 WHERE acc_no = 'A-101';
UPDATE account SET balance = balance + 500
 WHERE acc_no = 'B-2O2';   -- Updated Rows: 0

SELECT * FROM account
 WHERE acc_no IN ('A-101', 'B-202');
-- A-101 500 · B-202 2000  -> wrong!

ROLLBACK;   -- A-101 1000 again: nothing lost
```

Example 2 — Automatic rollback on an error

A CHECK constraint rejects Nila's negative balance.

```
BEGIN;
UPDATE account SET balance = balance + 1500
 WHERE acc_no = 'E-505';   -- succeeds
UPDATE account SET balance = balance - 1500
 WHERE acc_no = 'D-404';
-- ERROR [23514]: violates check
--   constraint "account_balance_check"

COMMIT;   -- no error message...
-- ...but nothing saved: D-404 500 · E-505 3000
-- A failed transaction turns COMMIT into
-- ROLLBACK; until then statements get [25P02]
```

TWO USERS AT THE SAME TIME

Isolation

Changes inside a transaction are invisible to everyone else until COMMIT. In DBeaver each SQL editor tab (Ctrl+]) is a separate session — just like a different user. Run the steps in order t1 → t8.

Tab 1 — You (t1–t4, t7)

Open a transaction and change Jamal's balance.

```
SET search_path TO txn_lab;
SELECT pg_backend_pid();   -- note this ID

-- t1
SELECT balance FROM account
 WHERE acc_no = 'C-303';   -- 1500
-- t2, t3
BEGIN;
UPDATE account SET balance = balance + 1000
 WHERE acc_no = 'C-303';
-- t4: same SELECT          -> 2500
-- t7
ROLLBACK;
```

Tab 2 — Another user (t5, t6, t8)

Watch the same row while Tab 1 is still open.

```
SET search_path TO txn_lab;

-- t5
SELECT balance FROM account
 WHERE acc_no = 'C-303';   -- still 1500

-- t6 / t8: who is inside a transaction?
SELECT pid, state,
       now() - xact_start AS open_for
  FROM pg_stat_activity
 WHERE pid = 12345;   -- Tab 1's ID
-- t6: 'idle in transaction'
-- t8: 'idle'  (Tab 1 has finished)
```

A BOOKMARK INSIDE A TRANSACTION

Savepoints

ROLLBACK undoes everything — even the work that was correct. A SAVEPOINT is a bookmark: ROLLBACK TO SAVEPOINT undoes only the work after it, and the transaction stays open.

Example 1 — Problem: ROLLBACK undoes all

1 laptop + 2 mice ordered; clerk types 20 mice.

```
BEGIN;
INSERT INTO orders (order_id, customer)
  VALUES (3, 'Rakib');
-- Item 1: laptop (correct)
INSERT INTO order_item (order_id, product_id,
  qty, unit_price) VALUES (3, 1, 1, 85000);
-- Item 2: mouse, WRONG quantity
INSERT INTO order_item (order_id, product_id,
  qty, unit_price) VALUES (3, 2, 20, 500);

ROLLBACK;   -- the only way out
-- order AND laptop are gone too (0 rows)
```

Example 2 — The fix: SAVEPOINT

Undo only the mouse line; keep the order and laptop.

```
BEGIN;
INSERT INTO orders (order_id, customer)
  VALUES (3, 'Rakib');
INSERT INTO order_item (order_id, product_id,
  qty, unit_price) VALUES (3, 1, 1, 85000);
SAVEPOINT after_laptop;            -- bookmark
INSERT INTO order_item (order_id, product_id,
  qty, unit_price) VALUES (3, 2, 20, 500);
ROLLBACK TO SAVEPOINT after_laptop;
-- laptop only; transaction is still open
-- re-insert the mouse with qty = 2, then
COMMIT;   -- laptop (1) + mouse (2)
```

REMOVING A BOOKMARK

RELEASE SAVEPOINT

RELEASE SAVEPOINT deletes the bookmark but keeps the changes made after it. It does NOT commit — the transaction is still open and can still be rolled back as a whole.

Example — RELEASE keeps the change

Product 5 starts with 100 units in stock.

```
BEGIN;
UPDATE product SET stock = stock + 10
 WHERE product_id = 5;    -- 100 -> 110
SAVEPOINT s1;
UPDATE product SET stock = stock + 10
 WHERE product_id = 5;    -- 110 -> 120
RELEASE SAVEPOINT s1;     -- keep change, drop mark
ROLLBACK TO SAVEPOINT s1;
-- ERROR [3B001]: savepoint "s1" does not exist
ROLLBACK;   -- undo the whole transaction
SELECT stock FROM product
 WHERE product_id = 5;    -- 100
```

Quick Reference

Every transaction command from this week.

```
SET search_path TO txn_lab;  -- every new tab

BEGIN;                       -- start a transaction
COMMIT;                      -- make it permanent
ROLLBACK;                    -- undo all since BEGIN

SAVEPOINT name;              -- place a bookmark
ROLLBACK TO SAVEPOINT name;
                             -- undo back to bookmark
RELEASE SAVEPOINT name;      -- remove the bookmark,
                             -- keep the changes
```

CHOOSING THE RIGHT ONE

Savepoint Commands Compared

Each command treats the work done after a savepoint — and the savepoint itself — differently.

Command	Changes after the savepoint	The savepoint itself
SAVEPOINT s	Nothing changes — a bookmark is placed	Created
ROLLBACK TO SAVEPOINT s	Undone — transaction stays open	Kept (can roll back to it again)
RELEASE SAVEPOINT s	Kept — but not committed yet	Removed
ROLLBACK	Everything since BEGIN undone	All removed
COMMIT	Everything saved permanently	All removed

LAB TASK · WEEK 7 · PART A

Restocking — Single Savepoint

`product(product_id PK, name, stock)` · `Initial row: (5, 'Keyboard', 50)`

1. **Setup** — Create the product table and insert the initial row for product_id 5.
2. **BEGIN** — Start a transaction and add the first batch of 10 units to product_id 5.
3. **SAVEPOINT** — Create a savepoint named s1 right after the first batch.
4. **Mistake** — Add the second batch of 10 units to product_id 5 (entered by mistake).
5. **Undo** — Use ROLLBACK TO SAVEPOINT s1 to undo only the second batch.
6. **COMMIT** — Save the transaction permanently, then display the final row of product_id 5.

LAB TASK · WEEK 7 · PART B

Restocking — Multiple Savepoints

`Continue from Part A's stock · BEGIN a new transaction · Show stock after each rollback`

1. **Small delivery** — Add 5 units to product_id 5, then create savepoint s1.
2. **Large delivery** — Add 15 units, then create savepoint s2.
3. **Sale** — Subtract 8 units (sold to a customer), then create savepoint s3.
4. **Wrong entry** — Add 20 units by mistake; undo only this with ROLLBACK TO SAVEPOINT s3.
5. **More undo** — Sale cancelled → ROLLBACK TO SAVEPOINT s2; large delivery wrong → ROLLBACK TO s1.
6. **COMMIT** — Save the transaction and display the final row of product_id 5.